

FLEXSIGHT

IoT Temperature Monitoring and Alert System

Holberton School — Final Portfolio Project

Hamsa Bniam Alammar

Project Manager · Hardware & Sensor
Integration

Rabea Thabit

SCM · Backend Developer

Munirah Alotaibi

QA · Frontend Developer · Documentation

Live project: <https://flexsight.dev/links>

GitHub: <https://github.com/rabea14180-max/Portfolio-Project>

Unnoticed Temperature Changes Cause Damage

- Sensitive environments can be damaged by temperature increases that go unnoticed.
- Manual, periodic checks are slow and do not provide continuous visibility.
- FlexSight provides centralized hourly monitoring, stored readings, and alerts.

Target Environments

Warehouses

Factories

Server Rooms

Data Centers

Electrical Rooms

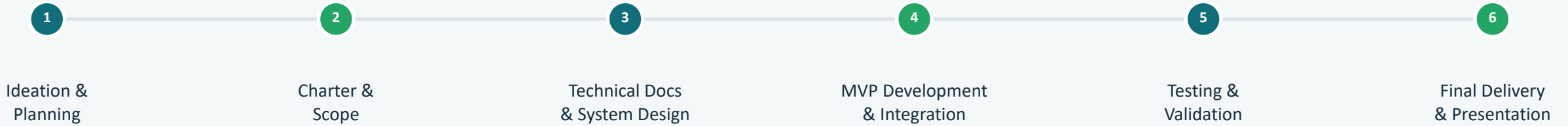
Technical Rooms

Offices

MVP Concept

A focused web dashboard that turns raw sensor readings into continuous visibility and timely alerts — for the people responsible for protecting sensitive spaces.

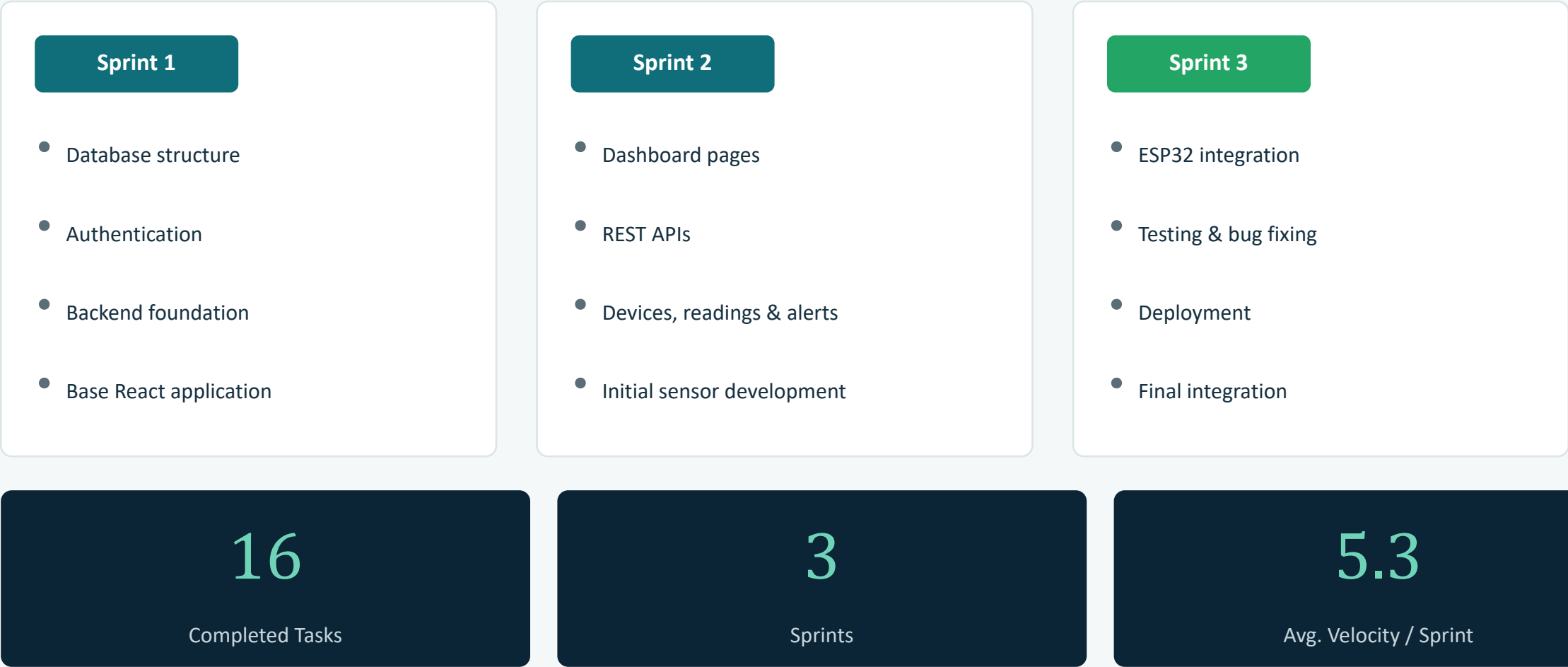
Project Journey



Key Decisions

- Focus only on temperature monitoring
- Use a web dashboard
- Use PostgreSQL for structured data
- Use role-based permissions
- Keep the MVP focused and realistic

Development Across Three Sprints



Technologies and Technical Decisions

FRONTEND

React & Vite

Fast development with a component-based, maintainable UI

DATABASE

PostgreSQL

Reliable, structured storage for devices, readings, alerts & users

SOURCE CONTROL

Git & GitHub

Version control and collaboration across three team members

BACKEND

Python & Flask

Lightweight framework well suited to a focused REST API

HARDWARE

ESP32 & DHT11

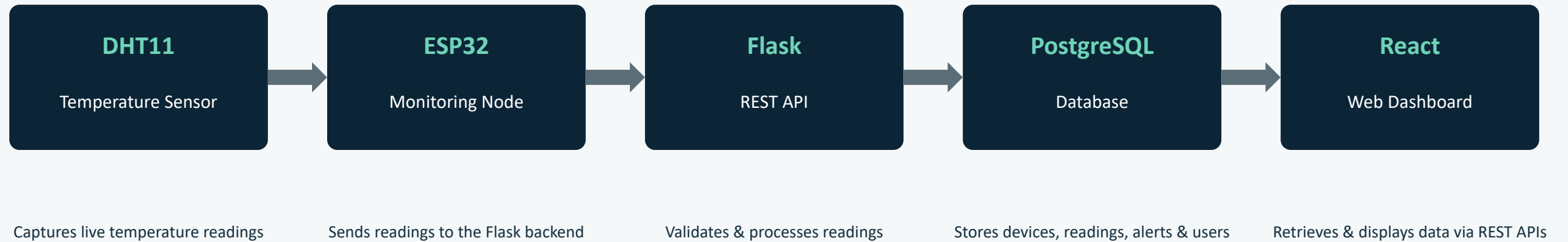
Affordable, reliable temperature sensing with Wi-Fi connectivity

DEPLOYMENT

Cloud Deployment

Keeps the working MVP accessible for review and demo

System Architecture



How Alerts Work

When the configured threshold is exceeded, the system creates a dashboard alert for the responsible users.

Core Features & Role-Based Access

Core Features

- Owner registration & secure login
- Dashboard with system statistics
- Device monitoring & status
- Stored, hourly temperature readings
- User-configurable temperature threshold
- Alert generation, severity & status
- Alert acknowledgement & resolution
- User management

Role-Based Access

	Owner	Admin / Manager	Inspector
Dashboard	Full	Full	View only
Readings	Full	Full	View only
Users & Devices	Manage	Manage	—
Alerts & Threshold	Manage	Manage	—
Settings (password)	Full	Full	Change password

Inspector has view-only access to the Dashboard and Readings, and can use Settings only to change their password.

Live MVP Demo

- 1

Open FlexSight and log in
- 2

Show the dashboard and system statistics
- 3

Show devices and device status
- 4

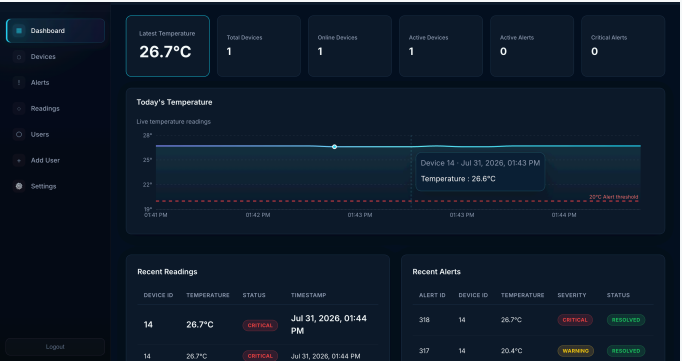
Show stored temperature readings
- 5

Change or explain the configurable threshold
- 6

Show the generated alert when the configured threshold is exceeded
- 7

Acknowledge or resolve an alert
- 8

Demonstrate role-based access



FlexSight Dashboard

The 'Temperature Readings' table includes a 'Filter Readings' section at the top with fields for Device ID (e.g., 1), Device Name (e.g., Serial Room Sensor), Start Date (dd/mm/yyyy), and End Date (dd/mm/yyyy). Below the filter is a table of temperature readings.

DEVICE ID	DEVICE NAME	TEMPERATURE	STATUS	TIMESTAMP	NAME
14	ESP32 Sensor	26.7°C	CRITICAL	Jul 31, 2026, 01:44 PM	rab2008
14	ESP32 Sensor	26.7°C	CRITICAL	Jul 31, 2026, 01:44 PM	rab2008
14	ESP32 Sensor	26.7°C	CRITICAL	Jul 31, 2026, 01:44 PM	rab2008
14	ESP32 Sensor	26.7°C	CRITICAL	Jul 31, 2026, 01:44 PM	rab2008
14	ESP32 Sensor	26.7°C	CRITICAL	Jul 31, 2026, 01:44 PM	rab2008
14	ESP32 Sensor	26.7°C	CRITICAL	Jul 31, 2026, 01:44 PM	rab2008
14	ESP32 Sensor	26.7°C	CRITICAL	Jul 31, 2026, 01:44 PM	rab2008
14	ESP32 Sensor	26.6°C	CRITICAL	Jul 31, 2026, 01:43 PM	rab2008
14	ESP32 Sensor	26.7°C	CRITICAL	Jul 31, 2026, 01:43 PM	rab2008

Temperature Readings

The 'All Alerts' table displays a list of alerts with columns for Alert ID, Device ID, Temperature, Severity, Status, Triggered At, Resolved By, and Actions.

ALERT ID	DEVICE ID	TEMPERATURE	SEVERITY	STATUS	TRIGGERED AT	RESOLVED BY	ACTIONS
318	14	26.7°C	CRITICAL	RESOLVED	Jul 31, 2026, 01:40 PM	rab2008	
317	14	20.4°C	WARNING	RESOLVED	Jul 23, 2026, 11:05 AM	—	
316	14	20.1°C	WARNING	RESOLVED	Jul 22, 2026, 03:10 PM	—	
315	14	20.1°C	WARNING	RESOLVED	Jul 22, 2026, 02:03 PM	—	
314	14	20.2°C	WARNING	RESOLVED	Jul 22, 2026, 12:12 PM	—	
313	14	26.2°C	CRITICAL	RESOLVED	Jul 21, 2026, 10:07 PM	—	
4	5	52°C	CRITICAL	RESOLVED	Jul 7, 2026, 03:52 PM	rab2008	
3	5	47°C	WARNING	RESOLVED	Jul 7, 2026, 03:52 PM	rab2008	

Alerts

Testing and Validation Results

38

Total Automated
Checks Passed

26

Frontend
Unit Tests

12

Backend &
Integration Tests

Frontend Unit Test Breakdown

- **13** StatusBadge tests
- **4** ProtectedRoute tests
- **9** API Helper tests

Coverage Highlights

- Stable web MVP deployed
- Full sensor-to-dashboard workflow integrated
- API health, authentication & dashboard data validated
- Devices, readings, alerts & users covered
- Connectivity & production integration verified

Challenges, Responses & Lessons Learned

Challenges

- Hardware and ESP32 integration
- Coordinating parallel frontend & backend work
- Testing the complete sensor-to-dashboard flow
- Keeping the MVP within the agreed scope

Responses

- Iterative sensor testing
- Clearer task ownership
- Incremental integration
- Early automated testing
- Focused scope decisions

Lessons Learned

- Begin hardware testing earlier
- Leave buffer time for integration
- Test small components before full integration
- Clear ownership improves team decisions

Team Contributions & Retrospective

Hamsa Bnian Alammar

Project Manager

- Led planning & sprint coordination
- Worked on the ESP32 and sensor hardware
- Supported hardware integration

Rabea Thabit

SCM · Backend Developer

- Managed SCM and repository structure
- Built the Flask backend and REST APIs
- Implemented PostgreSQL integration

Munirah Alotaibi

QA · Frontend Developer

- Built frontend views and components
- Planned and executed QA testing
- Prepared and maintained project documentation

What Went Well

- All 16 tasks were completed
- Clear role distribution supported parallel progress
- Testing caught issues before final delivery

What Could Improve

- Earlier hardware testing
- More frequent cross-team integration checks

CONCLUSION

Conclusion & Next Steps

- FlexSight delivered a stable, deployed IoT temperature-monitoring MVP.
- The system connects a real sensor to a Flask backend, PostgreSQL database, and React dashboard.
- The implemented MVP meets its agreed scope.
- The system supports user-configured thresholds, dashboard alerts, and role-based access.

Possible Future Improvements

(not currently implemented)

Mobile application

Additional sensor types

Advanced analytics and reporting

Multi-branch support

External system integrations

Live project: <https://flexsight.dev/>

GitHub: <https://github.com/rabea14180-max/Portfolio-Project>

Thank You